Lección 4: Sentencias de Control de Flujo en Python

Condicionales: If else, While

Made with GAMMA

Comparaciones

Las sentencias de control comienzan con una **comparación** que siempre devuelve un valor booleano (**True** o **False**). Estos operadores son fundamentales para que tu código pueda tomar decisiones inteligentes basadas en datos.

Operadores de Comparación

== (Igual a)

Comprueba si dos valores son exactamente iguales

```
edad == 18
```

!= (Diferente de)

Verifica que dos valores sean distintos

```
ciudad != 'Madrid'
```

> (Mayor que)

Evalúa si el valor izquierdo es mayor

```
ventas > 1000
```

< (Menor que)

Comprueba si el valor es menor

```
precio < 50
```

Operadores Lógicos para Combinar Condiciones

and

Ambas condiciones deben ser True para que el resultado sea True

or

Al menos una condición debe ser True para obtener True

not

Invierte el resultado booleano de True a False o viceversa

Made with GAMMA

Estructuras de Selección: if / elif / else

El **control de flujo** permite que tu programa ejecute bloques de código solo bajo ciertas condiciones. Es como dar instrucciones específicas: "si esto ocurre, haz aquello".

01

if (Si...)

Ejecuta un bloque de código **solo si** la condición evaluada es True

```
if puntuacion >= 90:
    print("Sobresaliente")
```

02

elif (Si no, si...)

Permite encadenar múltiples condiciones alternativas que se evalúan en orden secuencial

```
elif puntuacion >= 70:
    print("Notable")
```

03

else (Si no...)

El bloque que se ejecuta cuando **ninguna** de las condiciones anteriores fue True

```
else:
    print("Aprobado")
```

¡La Indentación es Clave!

Los bloques de código *dentro* de if, elif o else **deben** estar indentados con 4 espacios o 1 tabulador. Python utiliza esta sangría para definir qué código pertenece a cada bloque. Sin indentación correcta, ¡tu código no funcionará!

Made with GAMMA

Bucle while

El ciclo **while** ejecuta un bloque de código **mientras** una condición permanezca siendo True.

1

Sintaxis y Ejemplo

```
contador = 0
while contador < 5:
    print(contador)
    contador = contador + 1
```

El ciclo continuará ejecutándose hasta que el contador alcance el valor 5.

2

⚠️ ¡Cuidado con el Bucle Infinito!

Si la condición dentro de while **nunca** se vuelve False, el programa quedará atrapado ejecutando el ciclo indefinidamente. Asegúrate siempre de incluir una forma de modificar la condición para salir del bucle.

Uso en Análisis de Datos

Ideal para tareas que deben continuar hasta que se cumpla un umbral o criterio específico. Por ejemplo: seguir consultando una API hasta recibir una respuesta exitosa, o procesar datos hasta alcanzar un nivel de precisión deseado.

Made with GAMMA

Control de While

Las declaraciones de control de flujo en Python te permiten manejar la ejecución de tus ciclos `while` (y otros bucles que veremos más adelante), dándote la flexibilidad de interrumpirlos o saltar iteraciones específicas.

break

Utiliza `break` para terminar el ciclo **inmediatamente**. La ejecución del programa salta al primer código que se encuentra después del bucle.

```
i = 0
while i < 5:
    if i == 3:
        break  # Sale del bucle cuando i es 3
    print(i)
    i += 1
# Output: 0, 1, 2
```

continue

Utiliza `continue` para saltar la iteración actual y pasar a la **siguiente** iteración del bucle. El código restante en la iteración actual se omite.

```
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue # Salta el print cuando i es 3
    print(i)
# Output: 1, 2, 4, 5
```

Made with GAMMA

Resumen

Decisión

El bloque **if / elif / else** permite que tu código tome diferentes caminos según el valor de los datos analizados.

Control Condicional

El ciclo **while** permite la ejecución continua hasta que se cumpla un criterio específico o umbral deseado.

Made with GAMMA